AWS Serverless Architecture –Assignments

This document consolidates the graded AWS serverless assignments completed as part of the Serverless Architecture and Cloud Automation coursework. The assignments demonstrate practical use of AWS services, automation using AWS Lambda, and scripting with the Boto3 SDK. All solutions were manually implemented, tested, and validated.

Assignment 2: Automated S3 Bucket Cleanup

This assignment focused on automating the cleanup of objects stored in an Amazon S3 bucket. AWS Lambda was used to identify and delete files older than 30 days, helping optimize storage usage and reduce operational overhead.

An S3 bucket was created and populated with sample objects for testing. A Lambda function implemented using Python and Boto3 scanned the bucket, evaluated object age using the LastModified timestamp, and removed files exceeding the retention threshold. Execution results were validated through Lambda test runs and CloudWatch logs.

Assignment 3: Monitor Unencrypted S3 Buckets

This assignment addressed security monitoring by auditing Amazon S3 buckets for server-side encryption. AWS Lambda and the Boto3 SDK were used to programmatically detect buckets without explicit encryption configuration.

The solution involved listing all S3 buckets and checking their encryption settings using the get_bucket_encryption API. Buckets without an explicit encryption configuration were logged. Lambda execution results were reviewed using Amazon CloudWatch logs for validation.

Assignment 7: DynamoDB Item Change Alert Using Lambda and SNS

This assignment implemented an event-driven alerting mechanism using DynamoDB Streams, AWS Lambda, and Amazon SNS. The objective was to receive notifications whenever data within a DynamoDB table was inserted, modified, or deleted.

A DynamoDB table was created with Streams enabled to capture New and Old Images. A Lambda function processed these stream events and published detailed change notifications to an SNS topic. Email alerts were successfully delivered and verified.

Assignment 15: Implement a Log Cleaner for S3

The objective of this assignment was to automate deletion of outdated log files stored in an Amazon S3 bucket. AWS Lambda, S3, and Amazon EventBridge were used to implement a scheduled log cleanup process.

The Lambda function scans objects within the specified bucket, calculates object age based on the LastModified timestamp, and deletes log files older than 90 days. The function was tested manually and scheduled to run weekly using an EventBridge rule.

Conclusion

These assignments demonstrate hands-on experience with AWS serverless architecture, event-driven design, and cloud automation. All implementations were tested and validated using AWS-native monitoring and logging tools.

Assignment 2: Automated S3 Bucket Cleanup Using AWS Lambda and Boto3

Objective

The objective of this assignment is to automate the cleanup of files in an Amazon S3 bucket that are older than 30 days using AWS Lambda and Boto3. This helps in cost optimization and automated storage management.

AWS Services Used

Amazon S3

AWS Lambda

IAM

Boto3 (Python SDK)

Amazon CloudWatch

Implementation Steps

1. Created an S3 bucket named assignment2-s3-cleanup-rohan-vijay-mangate.
2. Uploaded sample files into the S3 bucket for testing.
3. Created an IAM role with AmazonS3FullAccess and attached it to Lambda.
4. Created a Lambda function using Python 3.12.
5. Implemented Python code using Boto3 to delete objects older than 30 days.
6. Manually tested the Lambda function using the AWS console.

Lambda Function Code Explanation

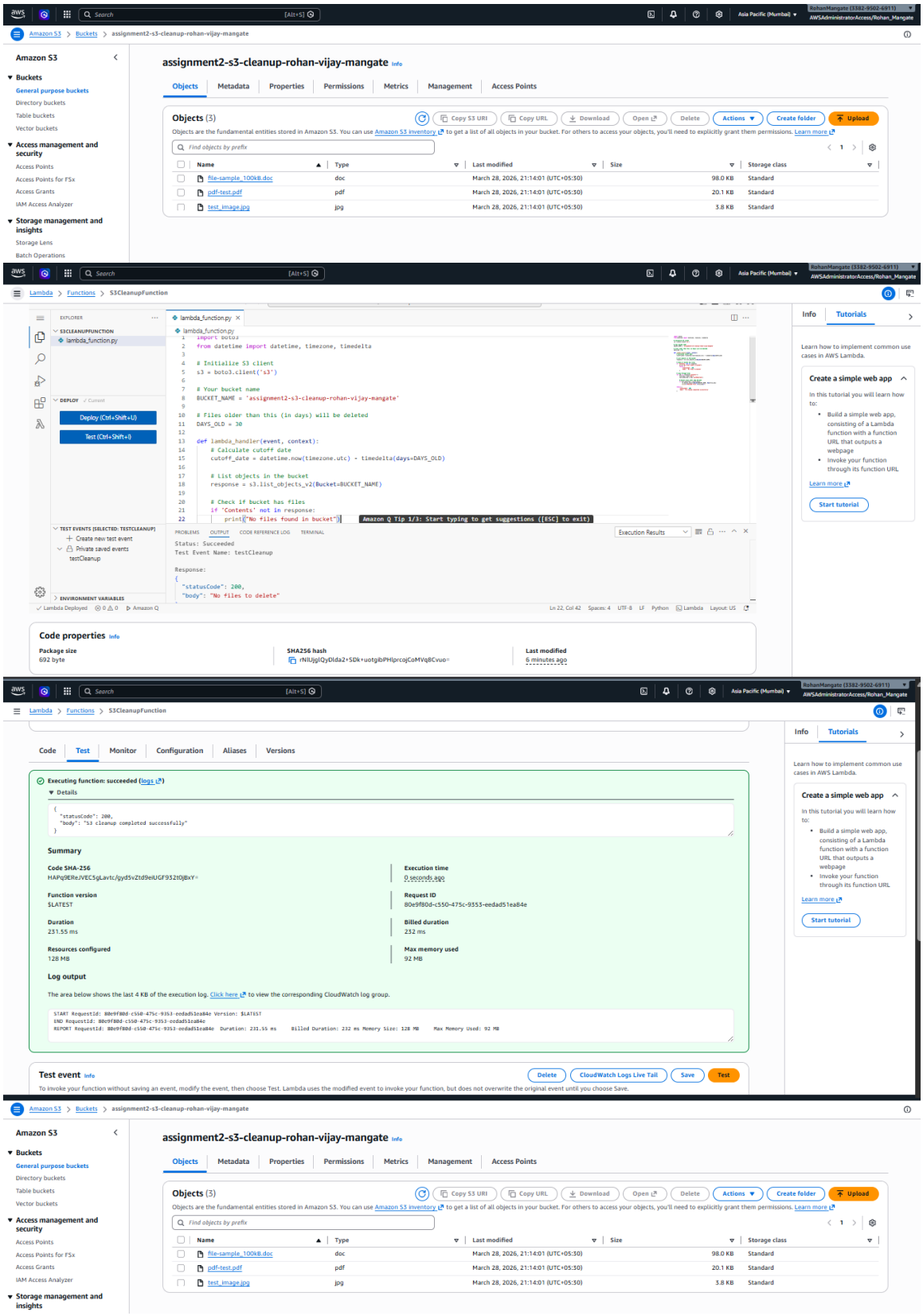
The Lambda function initializes an S3 client using Boto3, lists all objects in the bucket, checks their LastModified timestamp, and deletes files that are older than 30 days. Execution logs are printed to CloudWatch for verification.

Testing and Validation

The Lambda function was manually triggered using a test event. The execution completed successfully with a status code of 200, confirming that the cleanup logic works as expected.

Screenshots

The following screenshots are included for validation:
1. S3 bucket showing uploaded files before cleanup.
2. Lambda function Python code.
3. Lambda test execution result showing successful run.
4. CloudWatch logs and/or S3 bucket after cleanup.



Conclusion

This assignment demonstrates how AWS Lambda and Boto3 can be used to automate S3 bucket maintenance. The solution successfully fulfills the assignment objectives by implementing automated cleanup logic and validating it through manual testing.

Assignment 3: Monitor Unencrypted S3 Buckets Using AWS Lambda and Boto3

Objective

The objective of this assignment is to enhance AWS security by identifying Amazon S3 buckets that do not have explicit server-side encryption configurations enabled. This is implemented using AWS Lambda and the Boto3 Python SDK.

AWS Services Used

Amazon S3

AWS Lambda

IAM

Boto3 (Python SDK)

Amazon CloudWatch

Implementation Steps

1. Created an Amazon S3 bucket named unencrypted-bucket-rohan for encryption audit testing.
2. Created an IAM role with AmazonS3ReadOnlyAccess permission for Lambda execution.
3. Created a Lambda function using Python 3.12 runtime.
4. Implemented Boto3 logic to list all S3 buckets and check server-side encryption status.
5. Manually triggered the Lambda function to validate the encryption status of buckets.

Lambda Function Logic

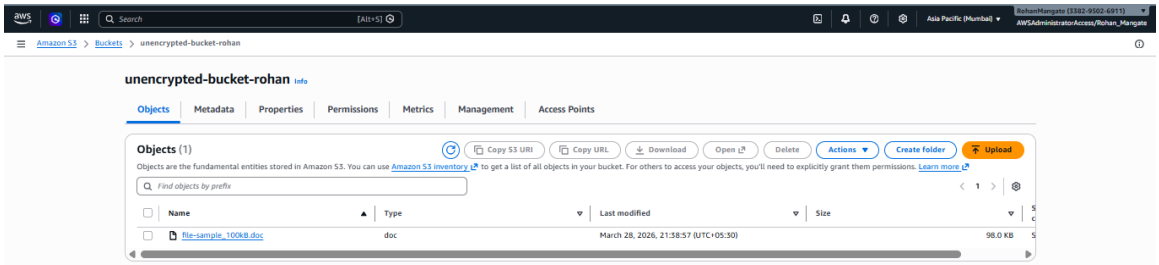
The Lambda function uses the boto3 S3 client to list all buckets within the AWS account. For each bucket, the function attempts to retrieve its encryption configuration using the get_bucket_encryption API. Buckets that do not return an encryption configuration are flagged and logged for review. Execution logs are written to Amazon CloudWatch.

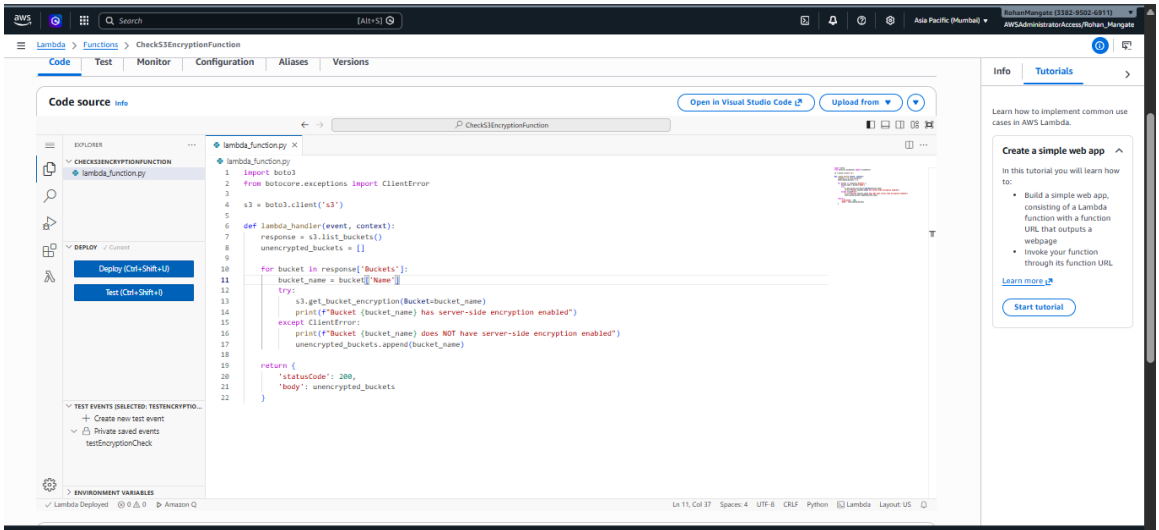
Testing and Validation

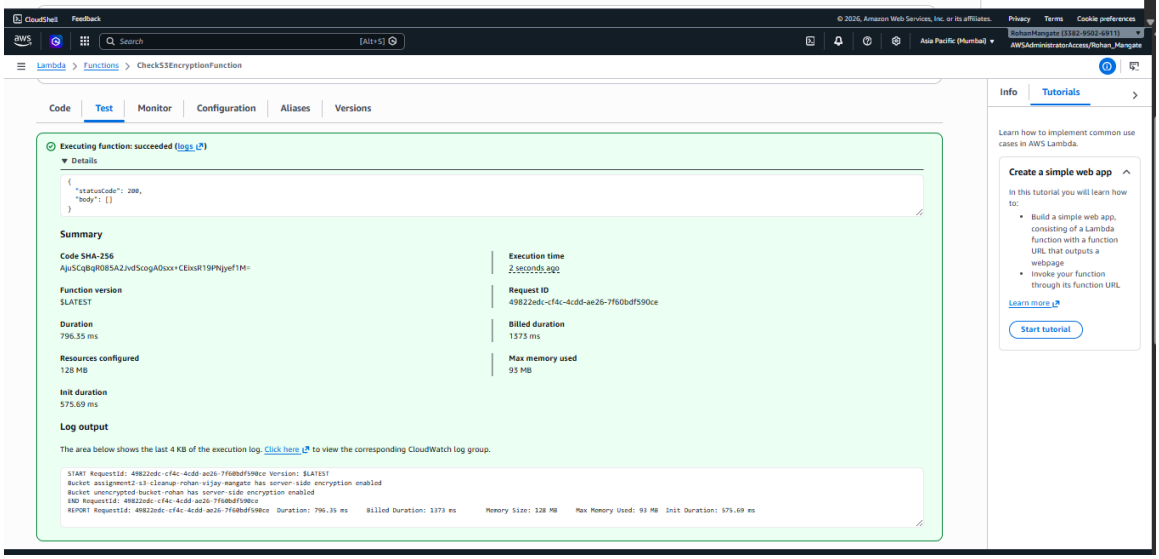
The Lambda function was manually invoked using a test event. The execution completed successfully and logged the encryption status of each S3 bucket. Due to AWS default settings, new buckets are automatically encrypted with SSE-S3, which was verified through logs.

Screenshots

The following screenshots are included as part of the assignment submission:
1. S3 bucket creation screen showing unencrypted-bucket-rohan.
2. Lambda function Python code for encryption check.
3. Lambda test execution result showing successful run.
4. CloudWatch logs displaying bucket encryption status.







Conclusion

This assignment successfully demonstrates how AWS Lambda and Boto3 can be used to audit Amazon S3 bucket encryption settings. The solution meets the security monitoring objective by programmatically validating server-side encryption and logging results for review.

Assignment 7: DynamoDB Change Alert with SNS

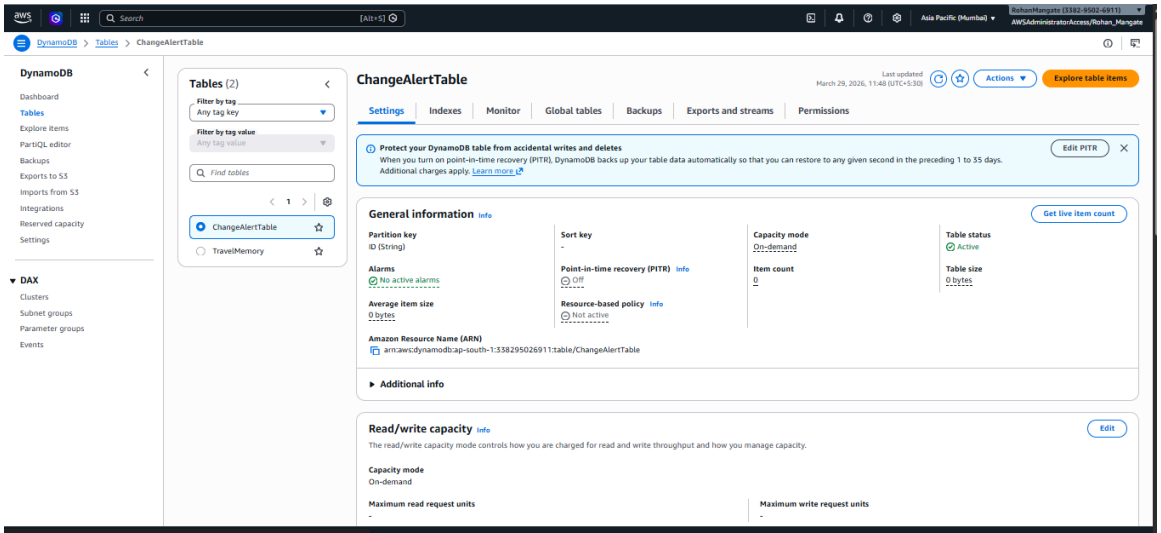
Objective:

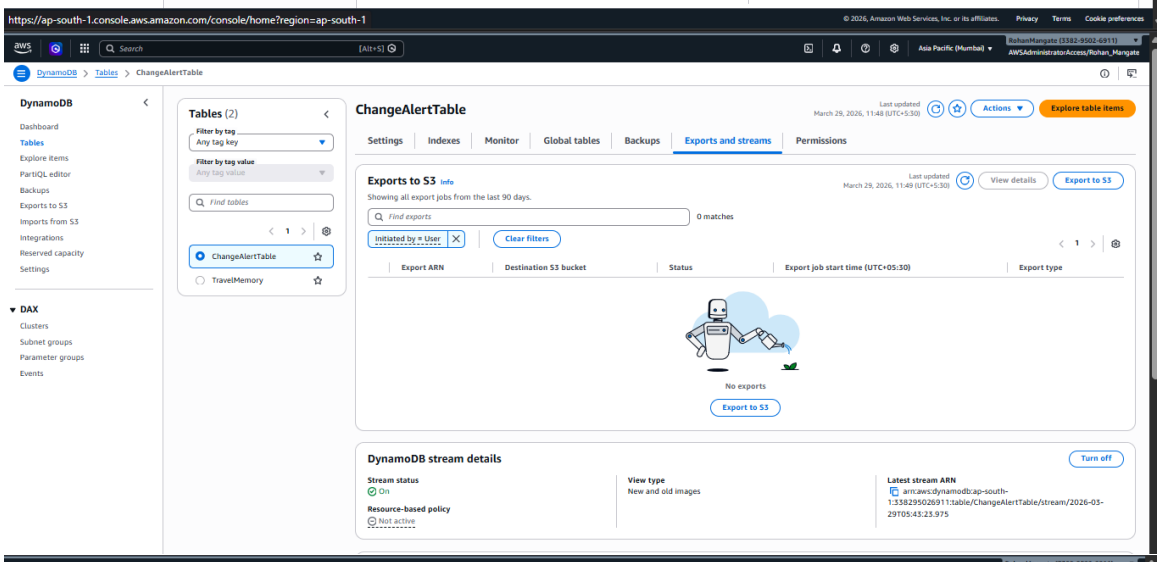
The objective of this assignment is to configure Amazon DynamoDB Streams with AWS Lambda to detect data changes (INSERT, MODIFY, DELETE) in a DynamoDB table and send real-time notifications using Amazon SNS email alerts.

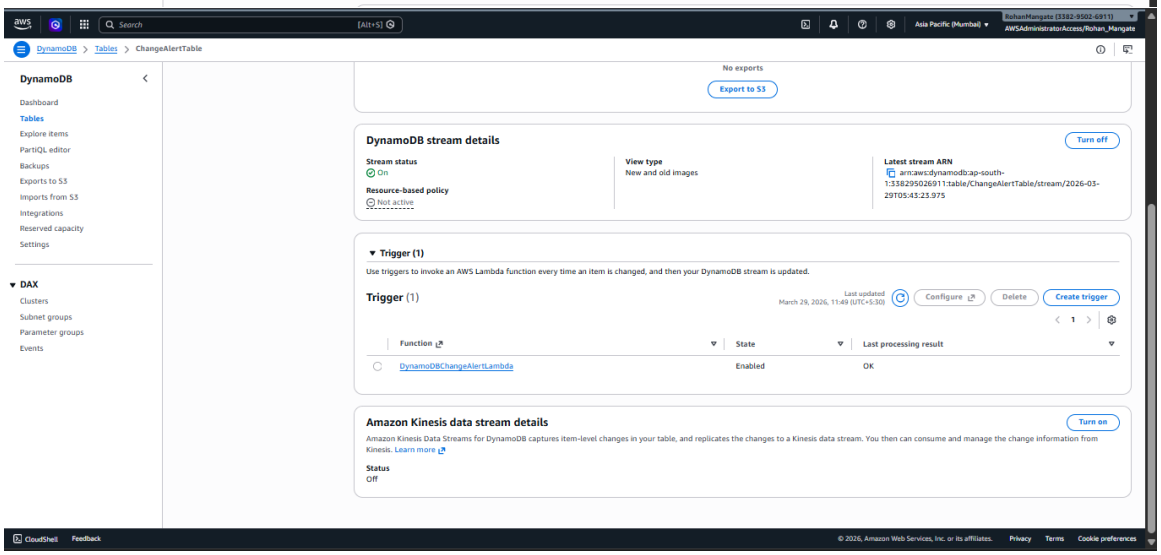
Step 1: DynamoDB Table Creation

A DynamoDB table named ChangeAlertTable was created with partition key id (String).
DynamoDB Streams were enabled with the view type set to New and Old Images.

Screenshot: DynamoDB table details and streams configuration.



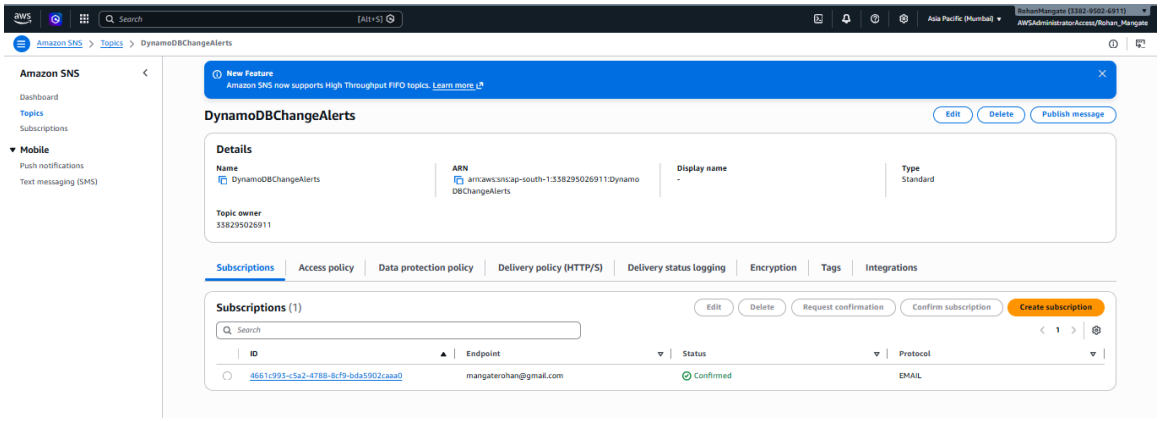




Step 2: SNS Topic and Subscription

An SNS topic named DynamoDBChangeAlerts was created.
An email subscription was added and successfully confirmed to receive notifications.

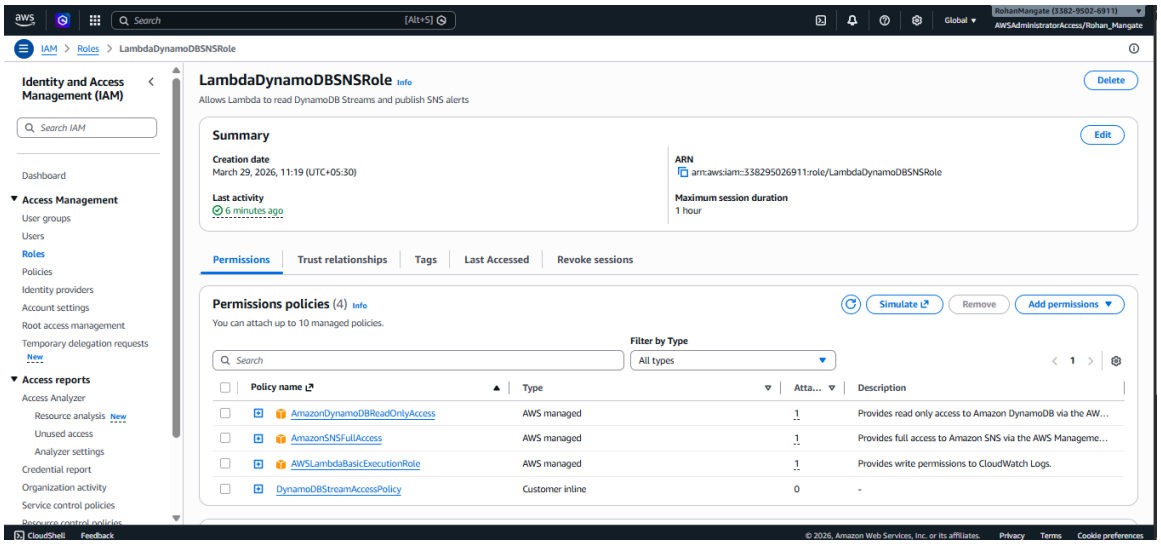
Screenshot: SNS topic details and confirmed subscription.



Step 3: IAM Role Creation

An IAM role named LambdaDynamoDBSNSRole was created for Lambda execution.
The role includes permissions for CloudWatch Logs, DynamoDB Streams access, and SNS publish actions.

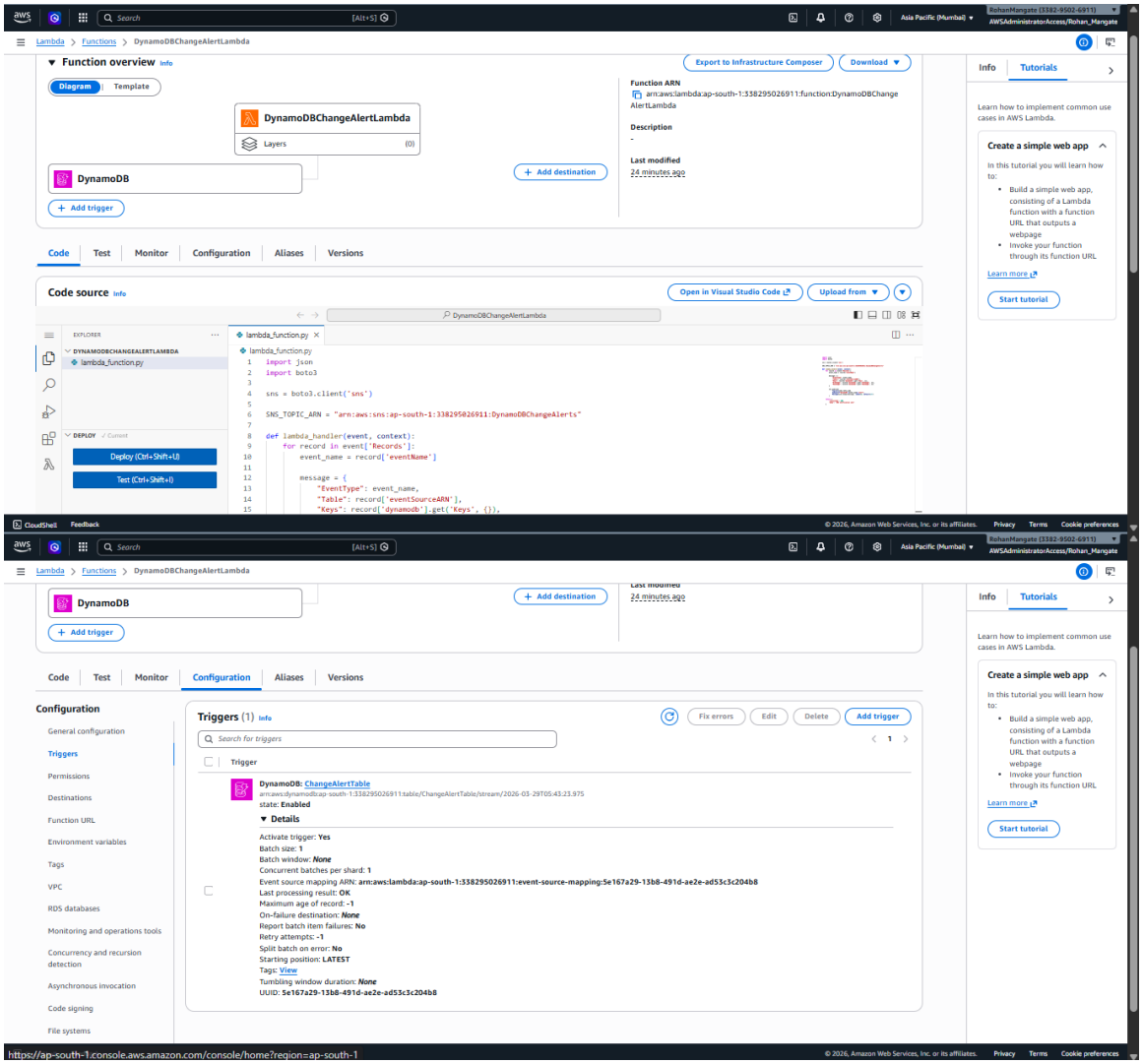
Screenshot: IAM role with attached policies.



Step 4: Lambda Function and Trigger Configuration

A Lambda function DynamoDBChangeAlertLambda was created using Python runtime.
The function was attached to the DynamoDB Stream trigger for ChangeAlertTable.

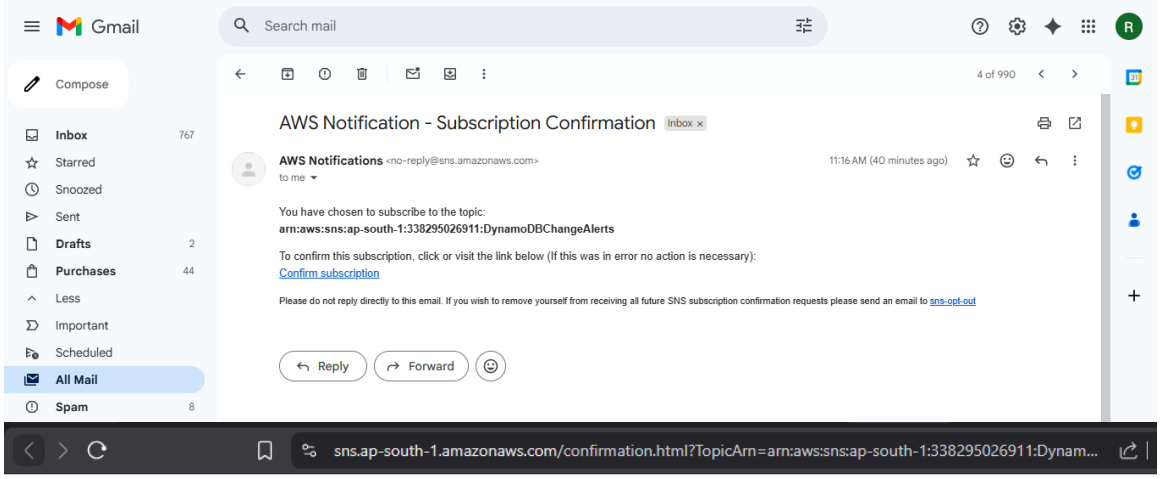
Screenshot: Lambda function details, trigger configuration, and function code.

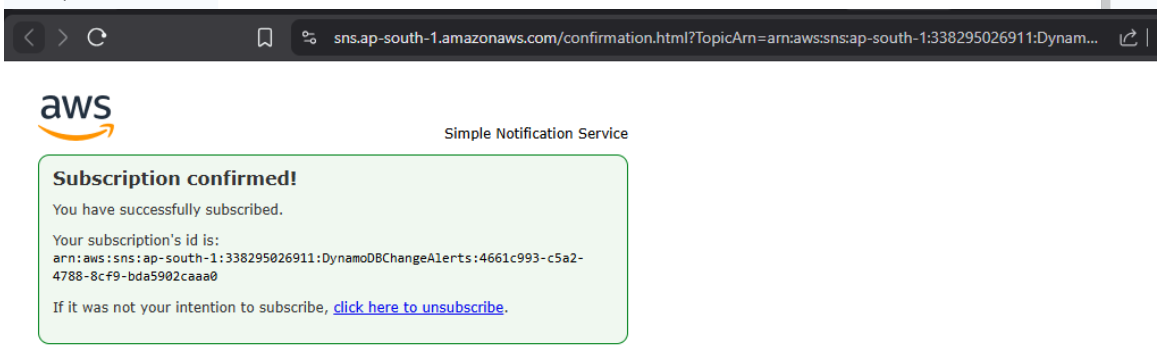


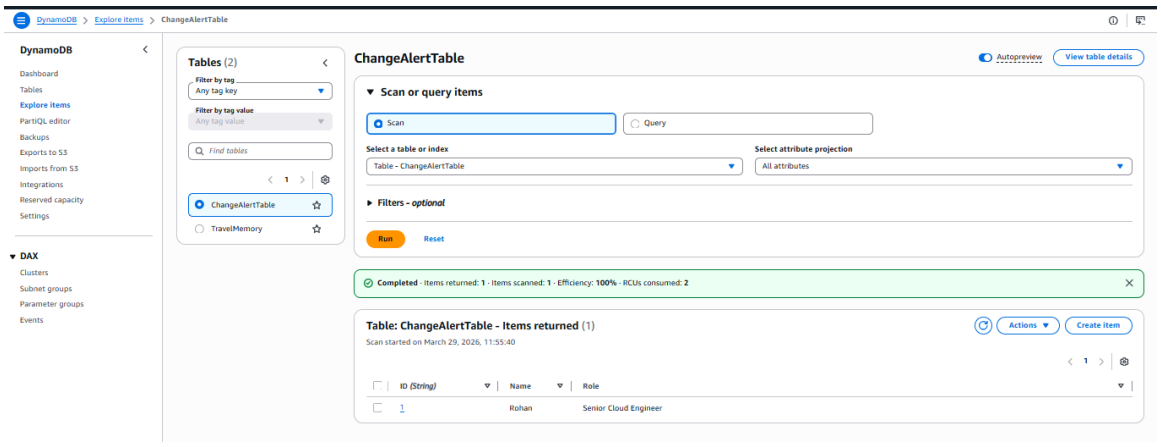
Step 5: Testing and Validation

Insert, Modify, and Delete operations were performed on the DynamoDB table.
Each operation successfully triggered the Lambda function and sent an SNS email alert.

Screenshots: DynamoDB operations and received email alerts.







Gmail

Search mail

Compose

Inbox 767
Starred
Snoozed
Sent
Drafts 2
Purchases 44

From | Any time | Has attachment | To | Is unread | Advanced search

1–50 of 990

AWS Notifications	Inbox	DynamoDB MODIFY Alert - { "EventType": "MODIFY", "Table": "arn:aws:dynamodb:ap-south-1:...	11:34 AM
AWS Notifications 3	Inbox	DynamoDB INSERT Alert - { "EventType": "INSERT", "Table": "arn:aws:dynamodb:ap-south-1:33...	11:33 AM
AWS Notifications 2	Inbox	DynamoDB REMOVE Alert - { "EventType": "REMOVE", "Table": "arn:aws:dynamodb:ap-south-1...	11:33 AM
AWS Notifications	Inbox	AWS Notification - Subscription Confirmation - You have chosen to subscribe to the topic: arn:...	11:16 AM

DynamoDB REMOVE Alert

Inbox ×

AWS Notifications <no-reply@sns.amazonaws.com>
to me

11:32 AM (24 minutes ago)


```
{
  "EventType": "REMOVE",
  "Table": "arn:aws:dynamodb:ap-south-1:338295026911:table/ChangeAlertTable/stream/2026-03-29T05:43:23.975",
  "Keys": {
    "ID": {
      "S": "1"
    }
  },
  "OldImage": {
    "ID": {
      "S": "1"
    }
  },
  "NewImage": {}
}
```


--
If you wish to stop receiving notifications from this topic, please click or visit the link below to unsubscribe:
https://sns.ap-south-1.amazonaws.com/unsubscribe.html?SubscriptionArn=arn:aws:sns:ap-south-1:338295026911:DynamoDBChangeAlerts:4661c993-c5a2-4788-8cf9-bda5902caaa0&Endpoint=mangaterohan@gmail.com

Please do not reply directly to this email. If you have any questions or comments regarding this email, please contact us at https://aws.amazon.com/support

AWS Notifications <no-reply@sns.amazonaws.com>
to me

11:33 AM (24 minutes ago)


```
{
  "EventType": "REMOVE",
  "Table": "arn:aws:dynamodb:ap-south-1:338295026911:table/ChangeAlertTable/stream/2026-03-29T05:43:23.975",
  "Keys": {
    "ID": {
      "S": "2"
    }
  },
  "OldImage": {
    "Role": {
      "S": "Cloud Engineer"
    },
    "ID": {
      "S": "2"
    },
    "Name": {
      "S": "Rohan"
```


...

Reply | Forward

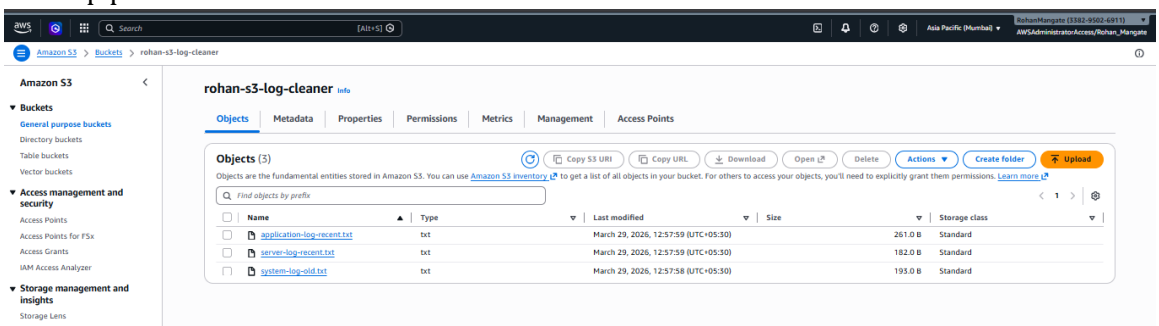
Assignment 15: Implement a Log Cleaner for S3

Objective

The objective of this assignment is to implement an automated log cleanup mechanism using AWS Lambda, Amazon S3, and Amazon EventBridge. The Lambda function periodically scans a specified S3 bucket and deletes log files older than 90 days, thereby optimizing storage usage and cost.

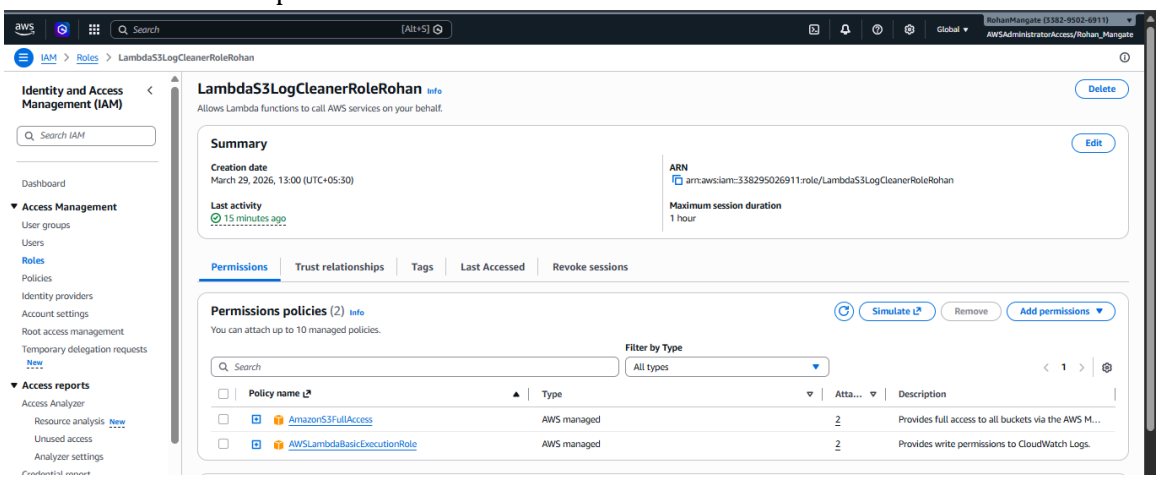
Step 1: S3 Bucket Creation and Log Upload

An S3 bucket was created to store application and system log files. Dummy log files were uploaded to simulate real-world logging data. These files serve as test inputs for the log cleanup process.



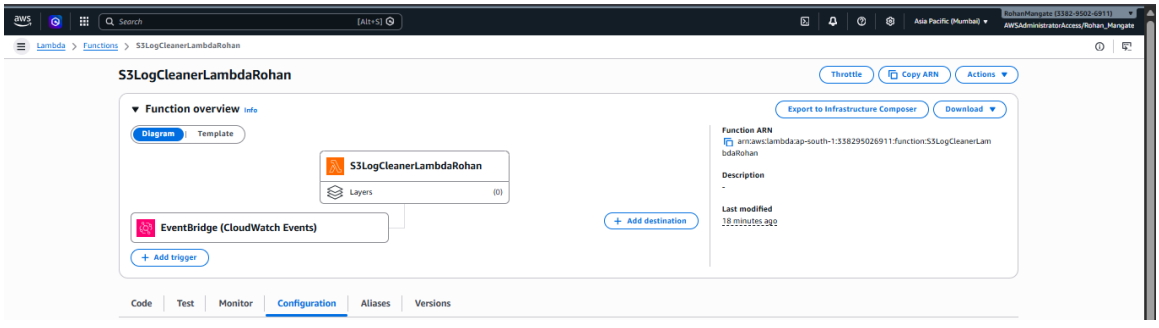
Step 2: IAM Role Creation for Lambda

An IAM role was created to grant the Lambda function permissions to access S3 and write logs to CloudWatch. The role was attached with the AWSLambdaBasicExecutionRole and AmazonS3FullAccess policies.



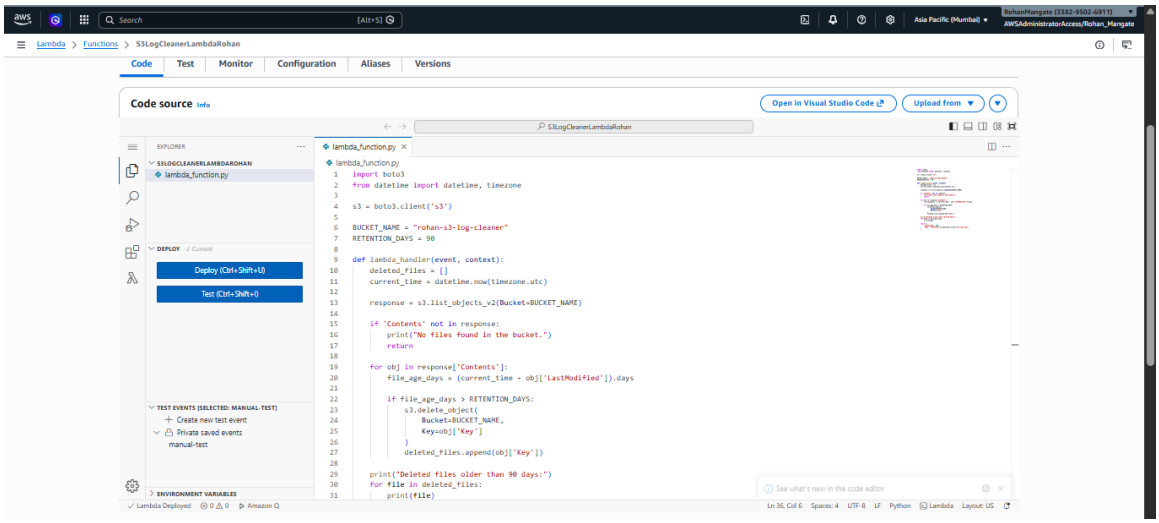
Step 3: Lambda Function Creation

A Lambda function was created using the Python runtime and assigned the previously created IAM role. This function contains the logic to identify and delete log files older than 90 days from the S3 bucket.



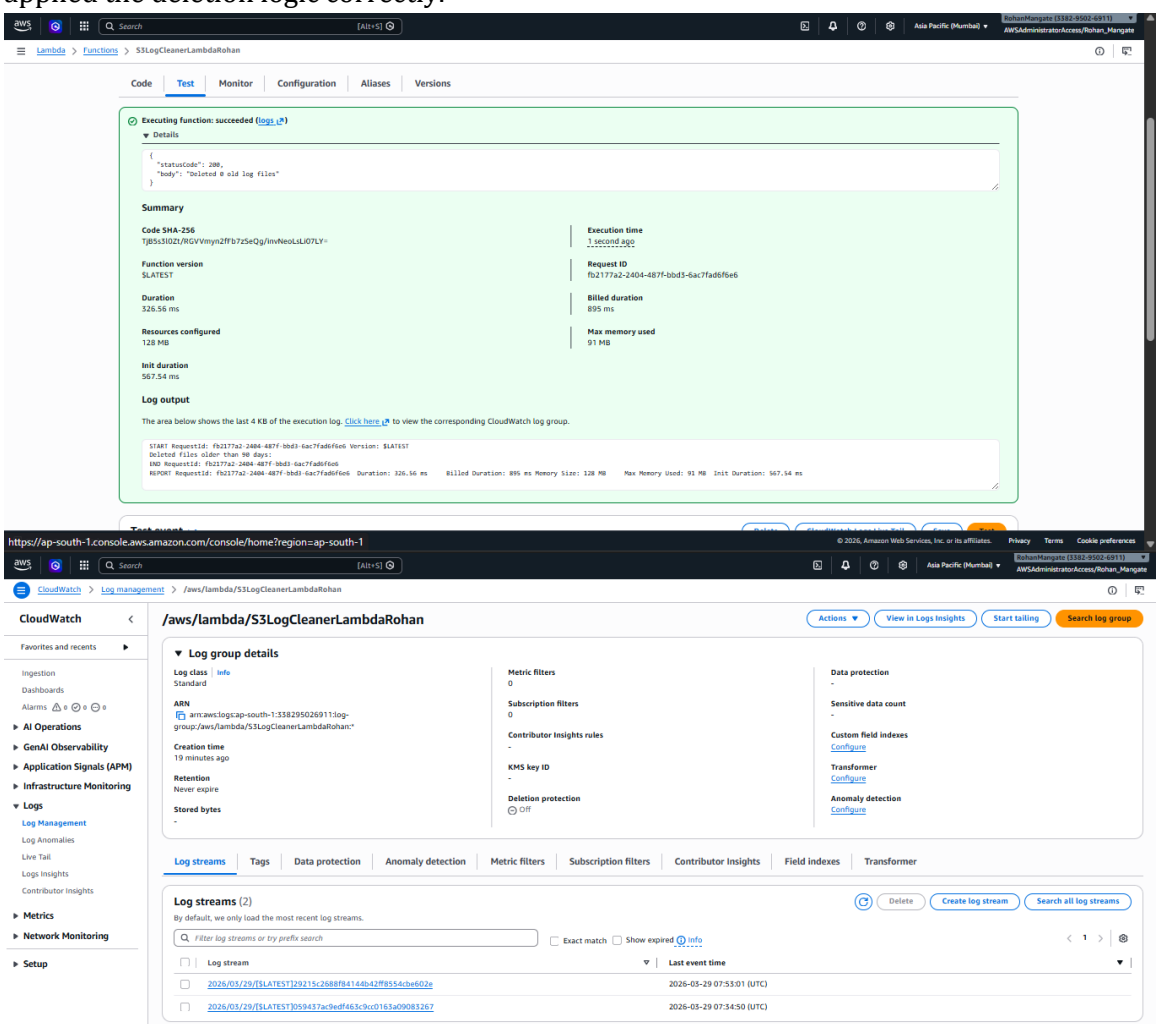
Step 4: Lambda Function Code Implementation

The Lambda function was implemented using Boto3. The code lists all objects in the S3 bucket, calculates the age of each file based on the LastModified timestamp, and deletes files exceeding the defined retention period of 90 days.



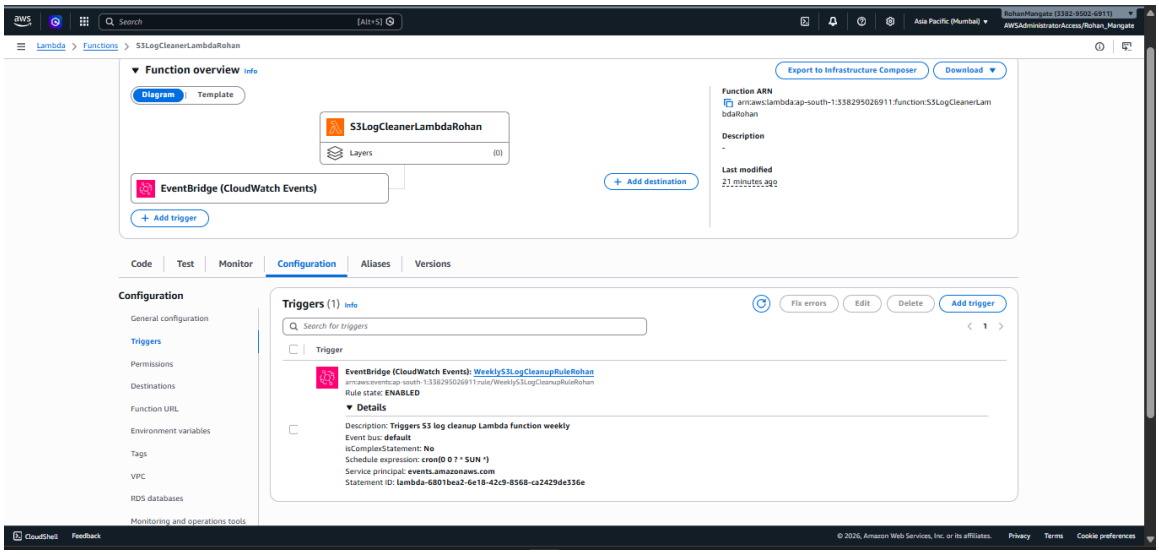
Step 5: Lambda Function Testing and Validation

The Lambda function was manually invoked using a test event. Execution logs were reviewed in Amazon CloudWatch to verify that the function executed successfully and applied the deletion logic correctly.



Step 6: EventBridge Scheduling

An Amazon EventBridge rule was created and attached to the Lambda function to trigger execution on a weekly schedule. The rule is enabled by default, ensuring automatic and periodic log cleanup.



Conclusion

This assignment successfully demonstrates the use of AWS Lambda and EventBridge to automate S3 log cleanup. The solution ensures efficient storage management by deleting outdated log files without manual intervention.